

ATC Timetable Management System

[models/manualTimetableModel.js](#) — Fully Explained

Every code block paired with a plain-language explanation and a concrete input → output example, using one consistent running scenario throughout.

Codebase: `refactoredCodes` | Date: 19 August 2026

The running example used throughout this document

A tmaster is placing the subject **"Cloud Computing"** (subject_id 41, taught by tutor **Japhet Kaijage**, program_type "full-time", 6.00 of 10.00 hours already scheduled) into venue **"Physics Lab"** (venue_id 65), on **MONDAY**, starting at the **07:30-08:15** slot. These are real values pulled from the live production database, not invented placeholders — the numbers stay consistent from the very first code block to the last.

Part 1 — SLOT_RANGES: the master time-table

```
const SLOT_RANGES = {
  "full-time": [
    "07:30-08:15", "08:15-09:00", "09:05-09:50", "09:50-10:35", "11:00-11:45",
    "11:45-12:30", "13:15-14:00", "14:00-14:45", "14:50-15:35", "15:35-16:20",
    "16:25-17:10", "17:10-17:55"
  ],
  "evening": [
    "16:25-17:10", "17:10-17:55", "18:00-18:45", "18:45-19:30",
    "19:35-20:20", "20:20-21:05", "21:10-21:55", "21:55-22:40"
  ],
  "VETA": [ /* same 12 slots as full-time */ ],
  "full-evening": ["16:25-17:10", "17:10-17:55"]
};
```

Four separate "allowed hours" lists, one per kind of student program. full-time and VETA students can only be scheduled 07:30-17:55 (12 slots, the normal working day). evening students can only be scheduled 16:25-22:40 (8 slots, late afternoon onward). full-evening is the narrow overlap window (just 2 slots, 16:25-17:55) for students who are both. This is a plain JavaScript object used as a lookup table — no database involved.

EXAMPLE

```
SLOT_RANGES["full-time"].includes("07:30-08:15")
→ true   (it is the very first entry in the full-time list)

SLOT_RANGES["evening"].includes("07:30-08:15")
→ false  (07:30-08:15 does not appear anywhere in the evening list)
```

Part 2 — programSlotMatch: the yes/no compatibility check

```
const programSlotMatch = (programType, slotTime) => {
  if (!programType || !slotTime) return false;
  const types = programType.split("+").map(t => t.trim().toLowerCase());

  if (types.includes("full-time") && !types.includes("evening")) {
    return SLOT_RANGES["full-time"].includes(slotTime);
  }
  if (types.includes("evening") && !types.includes("full-time")) {
    return SLOT_RANGES["evening"].includes(slotTime);
  }
  if (types.includes("full-time") && types.includes("evening")) {
    return SLOT_RANGES["full-evening"].includes(slotTime) || SLOT_RANGES["evening"].includes(slotTime);
  }
  if (types.includes("veta")) {
    return SLOT_RANGES["VETA"].includes(slotTime);
  }
  return false;
};
```

This function answers one question: "is slotTime a legal time for a student of programType?" — true or false, nothing else. It splits programType on "+" (a subject can be a combined value like "full-time+evening"), lowercases each piece, then checks which of the four SLOT_RANGES lists applies. Pure logic — no database access, so it can be called as many times as needed for free.

INPUT / OUTPUT EXAMPLES

```
programSlotMatch("full-time", "07:30-08:15")
  types = ["full-time"] → matches branch 1 → SLOT_RANGES["full-time"].includes("07:30-08:15")
  → true

programSlotMatch("evening", "07:30-08:15")
  types = ["evening"] → matches branch 2 → SLOT_RANGES["evening"].includes("07:30-08:15")
  → false

programSlotMatch("full-time+evening", "17:10-17:55")
  types = ["full-time","evening"] → matches branch 3
  → true (17:10-17:55 is in the full-evening overlap list)
```

Part 3 — resolveSlotColumn: translating a clicked time into a real column name

The venues table does not store "a list of available times" — it stores one column PER slot PER day. Monday alone has 18 separate columns: `monday_slot1_status` ... `monday_slot18_status`. This function's whole job is figuring out which ONE of those 18 columns corresponds to the time the user clicked in the UI.

```
async function resolveSlotColumn({ day, slot, venue_id }) {
  const dayLower = day.toLowerCase();
  const [cols] = await db.query(`SHOW COLUMNS FROM venues`);
  const colNames = cols.map(c => c.Field);
  const candidates = colNames.filter(c => c.startsWith(dayLower) && c.endsWith("_status")).sort();
  ...
  if (typeof slot === "string") {
    for (const statusCol of candidates) {
      const timeCol = statusCol.replace(/_status$/, "");
      const [rows] = await db.query(
        `SELECT \`${timeCol}\` AS timeVal FROM venues WHERE venue_id=? LIMIT 1`, [venue_id]
      );
      if (rows.length && rows[0].timeVal.trim() === slot.trim()) {
        return statusCol;
      }
    }
    throw new Error(`Time slot "${slot}" not found ...`);
  }
}
```

INPUT

```
resolveSlotColumn({ day: "MONDAY", slot: "07:30-08:15", venue_id: 65 })
```

STEP-BY-STEP

- 1) `SHOW COLUMNS FROM venues` → hundreds of column names returned
- 2) filter to `day="monday"` + ends with `"_status"` →
candidates = [`monday_slot1_status`, `monday_slot10_status`, `monday_slot11_status`, ..., `monday_slot2_status`, `monday_slot3_status`, ... , `monday_slot9_status`]
(alphabetically sorted – `slot10` comes right after `slot1`, not after `slot9`)
- 3) try `"monday_slot1_status"` first:
`SELECT monday_slot1 FROM venues WHERE venue_id=65` → `"07:30-08:15"`
`"07:30-08:15" === "07:30-08:15"` → MATCH on the very first try

OUTPUT

```
return "monday_slot1_status"
```

Part 4 — addtimetable: setup and validation (runs once per request)

```
export const addtimetable = async ({ day, venue_id, subject_ids, slot, logs = [] }) => {
  const log = (msg) => { logs.push(msg); console.log(msg); };

  if (!day || !venue_id || !Array.isArray(subject_ids) || subject_ids.length === 0 || !slot) {
    throw new Error("Missing required parameters: day, venue_id, subject_ids (array), slot.");
  }

  const dayUpper = day.toUpperCase();
  const dayToArrange = { MONDAY:1, TUESDAY:2, ..., SUNDAY:7 };
  const arrange = dayToArrange[dayUpper] || 0;

  const [venueDataRes] = await db.query(`SELECT * FROM venues WHERE venue_id = ?`, [venue_id]);
  if (!venueDataRes.length) { log(`■ Venue ID ${venue_id} not found.`); return; }
  const V = venueDataRes[0];

  const currentStatusCol = await resolveSlotColumn({ day, slot, venue_id });
  const currentTimeCol = currentStatusCol.replace(/_status$/, "");
  const finalCurrentSlot = currentSlotTime || slot;

  const slotNumMatch = currentStatusCol.match(/slot(\d+)_status$/);
  const currentSlotNum = parseInt(slotNumMatch[1]);
  // everything below this line runs ONCE PER SUBJECT, in a loop
}
```

INPUT (the whole function call)

```
addtimetable({
  day: "MONDAY", venue_id: "65", subject_ids: ["41"],
  slot: "07:30-08:15", logs: []
})
```

AFTER THIS SECTION RUNS

```
dayUpper      = "MONDAY"
arrange       = 1                      (Monday is day #1 of the week)
V             = { venue_id:65, venue_name:"Physics Lab", capacity:50, type:"Lab", ... }
currentStatusCol= "monday_slot1_status"
currentTimeCol = "monday_slot1"
finalCurrentSlot= "07:30-08:15"
currentSlotNum = 1
```

Part 5 — the per-subject loop begins: eligibility check

```

for (const subject_id of subject_ids) {
  const [subjectData] = await db.query(`
    SELECT s.*, u.full_name, u.user_id AS tutor_db_id
    FROM subjects s JOIN users u ON s.user_id = u.user_id
    WHERE s.subject_id = ?
  `, [subject_id]);

  if (!subjectData.length) { log(`■ No data found for subject_id = ${subject_id}.`); continue; }
  const S = subjectData[0];

  const currentLtpa = Number(S.ltpa || 0);
  const totalHours = Number(S.total_hours_per_week || 0);
  const remainingHours = totalHours - currentLtpa;

  if (remainingHours < 1.5) {
    log(`■ Subject ${subject_id} has insufficient remaining hours. Skipping.`);
    continue;
  }
}

```

INPUT

subject_id = "41"

QUERY RESULT (S)

```

S = {
  subject_id: 41, title: "Cloud Computing", subject_code: "CDT 05212",
  ltpa: 6.00, total_hours_per_week: 10.00, program_type: "full-time",
  semester: "II", full_name: "Japhet Kaijage", tutor_db_id: 187, ...
}

```

CALCULATION

```

currentLtpa    = 6.00
totalHours     = 10.00
remainingHours = 10.00 - 6.00 = 4.00
4.00 < 1.5 ?   → false   → DOES NOT skip; keeps going

```

Part 6 — finding the second half-slot (building the 1.5-hour class)

One class period = two consecutive 45-minute slots. The user only picked ONE (07:30-08:15). This section tries to find and validate the slot right after it (08:15-09:00).

```
let slotInfos = [ currentSlotInfo ]; // the slot the user picked goes in first
let canAssignDouble = true;

if (currentSlotNum >= 18) {
  canAssignDouble = false;
} else {
  const nextSlotNum = currentSlotNum + 1;
  const nextStatusCol = currentStatusCol.replace(`slot${currentSlotNum}_status`, `slot${nextSlotNum}_status`);
  const [nrows] = await db.query(`SELECT \`${nextTimeCol}\` AS t FROM venues WHERE venue_id=? LIMIT 1`, [venueId]);
  const nextSlotTime = nrows[0]?.t;

  const isFriday = dayUpper === "FRIDAY";
  if (isFriday && (slot numbers 7 or 8 involved)) canAssignDouble = false;

  if (nextSlotTime && !programSlotMatch(S.program_type, nextSlotTime)) canAssignDouble = false;

  if (canAssignDouble && nextSlotTime) {
    slotInfos.push({ statusCol: nextStatusCol, timeCol: nextTimeCol, slotTime: nextSlotTime });
  }
}

if (!canAssignDouble || slotInfos.length < 2) { log(...); continue; }
```

STEP-BY-STEP

```
currentSlotNum = 1 → not >= 18, so continue
nextSlotNum      = 2
nextStatusCol    = "monday_slot2_status"
query monday_slot2 for venue 65 → "08:15-09:00"
isFriday = false (day is MONDAY) → break-window check skipped, no penalty
programSlotMatch("full-time", "08:15-09:00") → true (in the full-time list)
→ canAssignDouble stays true
```

OUTPUT

```
slotInfos = [
  { statusCol: "monday_slot1_status", slotTime: "07:30-08:15" },
  { statusCol: "monday_slot2_status", slotTime: "08:15-09:00" }
]
slotInfos.length = 2 → proceed to the collision checks (does NOT skip this subject)
```

Part 7 — the collision checks: "is anyone else using either slot?"

```
let canAssign = true;
const reasons = [];

for (const sInfo of slotInfos) { // runs twice: once per half-slot
  const [existingEntries] = await db.query(`
    SELECT * FROM extracted_timetables WHERE day = ? AND slot = ? AND semester = ?
  `, [day, sInfo.slotTime, S.semester]);

  const tutorConflict = existingEntries.some(e => Number(e.created_by) === Number(S.tutor_db_id));
  const venueConflict = existingEntries.some(e => Number(e.venue_id) === Number(venue_id));

  const newParts = S.program_code.split("+").map(p => p.trim().toLowerCase());
  const programConflict = existingEntries.some(e =>
    e.program_code && e.program_code.split("+").map(p=>p.trim().toLowerCase())
      .some(p => newParts.includes(p))
  );

  if (tutorConflict || venueConflict || programConflict || !programSlotMatch(S.program_type, sInfo.slotTime))
    canAssign = false;
}

if (!canAssign) { log("■ COLLISION DETECTED..."); continue; }
```

QUERY 1 (for 07:30-08:15)

```
SELECT * FROM extracted_timetables
WHERE day="MONDAY" AND slot="07:30-08:15" AND semester="II"
→ [ ] (empty – nothing else is booked in this venue-agnostic slot+day+semester combo
      that happens to share this tutor, venue, or program)
```

QUERY 2 (for 08:15-09:00)

```
SELECT * FROM extracted_timetables
WHERE day="MONDAY" AND slot="08:15-09:00" AND semester="II"
→ [ ] (also empty)
```

OUTPUT

```
tutorConflict = false, venueConflict = false, programConflict = false
programSlotMatch passes for both slots
→ canAssign stays TRUE for both loop iterations → proceeds past this section
```

Contrast: if a DIFFERENT existing row had come back in Query 1 with created_by = 187 (Japhet Kaijage's own tutor_db_id), tutorConflict would be true, reasons would gain a line like "TUTOR COLLISION: Japhet Kaijage is already teaching on MONDAY 07:30-08:15", canAssign would become false, and this entire subject would be skipped — no rows would ever be written for it.

Part 8 — final hours check, then actually saving it

```
const ltpaIncrement = 1.5;
if (currentLtpa + ltpaIncrement > totalHours) { log("■ Would exceed total hours..."); continue; }

try {
  await db.query("START TRANSACTION");

  for (const sInfo of slotInfos) { // runs twice
    const [startTime, endTime] = sInfo.slotTime.split("-").map(t => t.trim());
    await db.query(`INSERT INTO extracted_timetables (...) VALUES (...)`, [ day, sInfo.slotTime,
      startTime, endTime, S.subject_code, S.title, ..., venue_id, V.venue_name, S.full_name, ...,
      S.tutor_db_id, new Date() ]);
    await db.query(`UPDATE venues SET \`${sInfo.statusCol}\` = 'used' WHERE venue_id = ?`, [venue_id]);
  }

  await db.query(`UPDATE subjects SET ltpa = COALESCE(ltpa, 0) + ? WHERE subject_id = ?`,
    [ltpaIncrement, subject_id]);

  await db.query("COMMIT");
  log(`✓ SUCCESS: Assigned subject ${subject_id} ...`);
} catch (insertErr) {
  await db.query("ROLLBACK");
  log(`■ Database error ...`);
}
```

CALCULATION

$6.00 + 1.5 = 7.50 \rightarrow 7.50 > 10.00 ? \rightarrow \text{false} \rightarrow \text{proceeds to save}$

DATABASE WRITES (inside one transaction)

```
INSERT INTO extracted_timetables (day, slot, start_time, end_time, subject_code, subject_name,
  venue_id, venue_name, tutor_name, program_name, semester, ...)
VALUES ("MONDAY", "07:30-08:15", "07:30", "08:15", "CDT 05212", "Cloud Computing",
  65, "Physics Lab", "Japhet Kaijage", "...", "II", ...) - row #1
```

```
INSERT INTO extracted_timetables (...)
VALUES ("MONDAY", "08:15-09:00", "08:15", "09:00", ... same subject ...) - row #2
```

```
UPDATE venues SET monday_slot1_status='used' WHERE venue_id=65
UPDATE venues SET monday_slot2_status='used' WHERE venue_id=65
UPDATE subjects SET ltpa = 6.00 + 1.5 = 7.50 WHERE subject_id=41
COMMIT
```

OUTPUT (what the tmaster sees on screen)

```
logs = [
  "Using status column: monday_slot1_status",
  "✓ SUCCESS: Assigned subject 41 (Cloud Computing) by Japhet Kaijage to MONDAY
  07:30-08:15 & 08:15-09:00 at Venue 65"
]
```

Part 9 — the two read functions, and the export

```
export const getTimetablesFromDB = async (filters) => {
  let query = `SELECT *, u.full_name AS tutor_name, ...
    FROM subjects s JOIN users u ON s.user_id = u.user_id
    WHERE (ltpa + 1.5) <= total_hours_per_week`;
  const params = [];
  if (filters.tutor_name) { query += ' AND u.full_name = ?'; params.push(filters.tutor_name); }
  if (filters.semester) { query += ' AND semester = ?'; params.push(filters.semester); }
  if (filters.program_type) { query += ' AND program_type = ?'; params.push(filters.program_type); }
  query += ' ORDER BY tutor_name ASC';
  const [rows] = await db.query(query, params);
  return rows;
};

export const getDistinctValues = async (column) => {
  const [rows] = await db.query(`SELECT DISTINCT ${column} FROM subjects s
    JOIN users u ON s.user_id = u.user_id WHERE (ltpa + 1.5) <= total_hours_per_week`);
  return rows;
};

export default { getTimetablesFromDB, getDistinctValues, addtimetable };
```

INPUT

```
getTimetablesFromDB({ tutor_name: "Japhet Kaijage", semester: "II", program_type: "" })
```

OUTPUT (real data shape)

```
[
  { subject_id: 9, subject_code: "ITU 08215/CSU 08216", title: "Undergraduate Project Implementation", ltpa: 0.00, total_hours_per_week: 2.00, tutor_name: "Japhet Kaijage" },
  { subject_id: 41, subject_code: "CDT 05212", title: "Cloud Computing", ltpa: 6.00, total_hours_per_week: 10.00, tutor_name: "Japhet Kaijage" },
  { subject_id: 47, subject_code: "ITU 07219", title: "Multimedia", ltpa: 6.75, total_hours_per_week: 10.00, tutor_name: "Japhet Kaijage" }
]
(note: after Part 8 ran, subject 41's ltpa in the database is now 7.50, not 6.00 — a fresh call to this function after that assignment would reflect the new value)
```

Note that `resolveSlotColumn` and `programSlotMatch` are NOT in the default export — they stay module-private, only usable from inside this file. Only `getTimetablesFromDB`, `getDistinctValues`, and `addtimetable` are visible to the outside (which is why the logic layer imports this whole file as one object, `timetableModel`, and calls `timetableModel.getTimetablesFromDB(...)` rather than a named import).